회원가입과 영화 평점 실습·필수 미션 수행 기록

검증 일시: 2026-11-03 17:38:46 (Asia/Seoul)
구현 스택: NestJS + MySQL / 실행 주소: http://localhost:3000

자동 테스트	실제 HTTP 검증	DB 확인
단위 1/1 · E2E 106/106	29/29 통과	bcrypt · UNIQUE · 삭제 확인

수행 내용

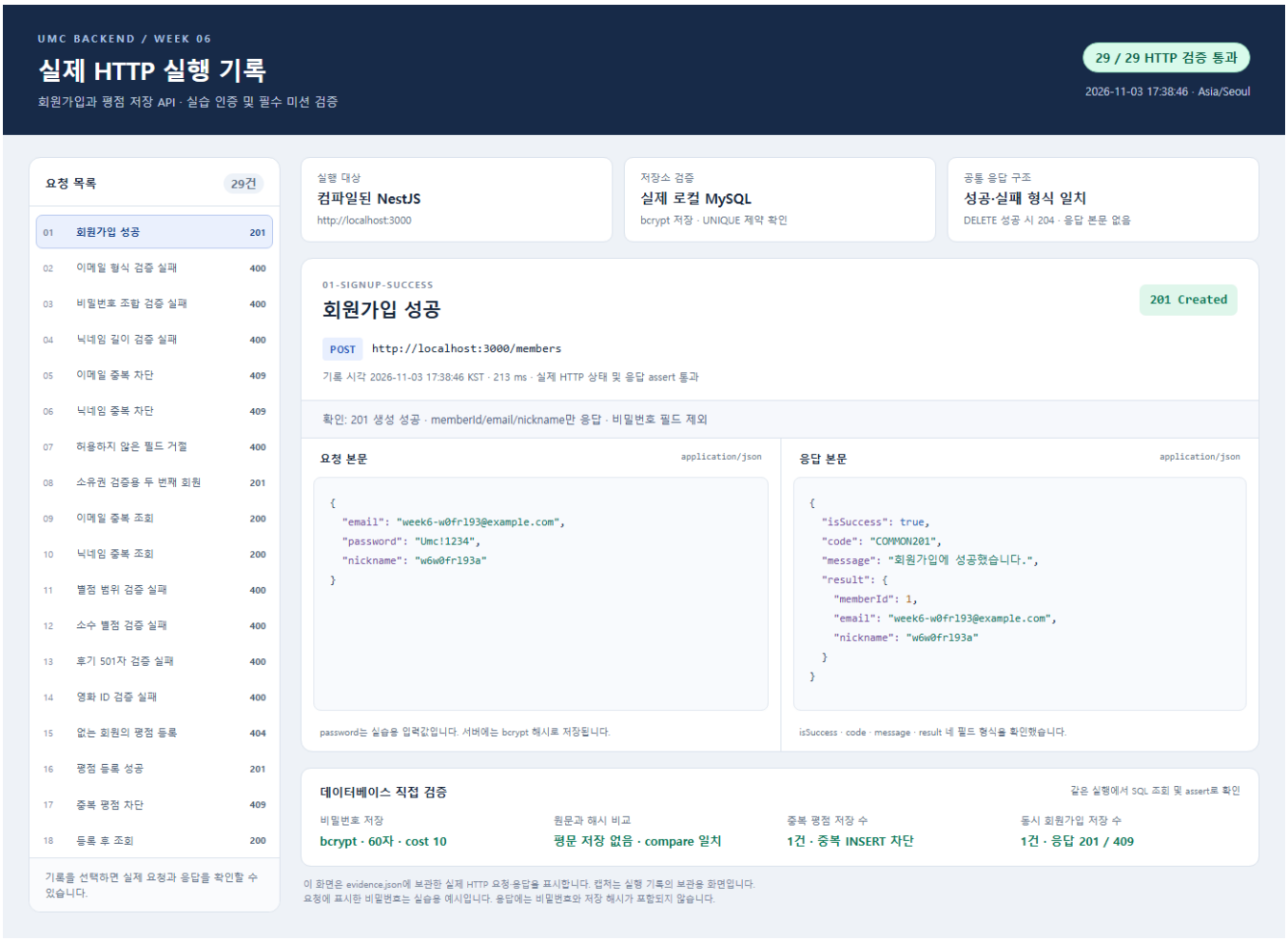
실습 1: 회원가입 DTO 검증, bcrypt cost 10 해시 저장, 공개 회원 정보만 반환.
실습 2: 전역 예외 필터로 성공·실패의 공통 응답 구조 적용.
실습 3 및 필수 미션: 평점 등록·수정·삭제, 소유자 확인, 중복·검증·없는 데이터 오류 처리.

제출 자료 구성

자료	프로젝트 내 위치
제출 가이드	docs/week6/SUBMISSION.md
API 명세	docs/week6/API.md
Postman 컬렉션 · 환경	docs/week6/week6.postman_*.json
실습 인증 및 미션 캡처	output/week6/screenshots/
실제 요청·응답 · DB 확인	output/week6/evidence.json
열어서 확인하는 실행 기록	output/week6/report.html
트러블슈팅 및 전후 증거	docs/week6/TROUBLESHOOTING.md

캡처는 실제 로컬 서버 요청을 실행한 뒤 원본 JSON을 화면으로 표시한 기록이다. Postman GUI 캡처로 표기하지 않았다. 예시 회원의 비밀번호는 실습용 값이며 응답·DB 원문 해시는 자료에 포함하지 않았다.

회원가입 성공



실제 HTTP 실행 기록 화면 | 2026-11-03 17:38:46 (Asia/Seoul)

확인 결과

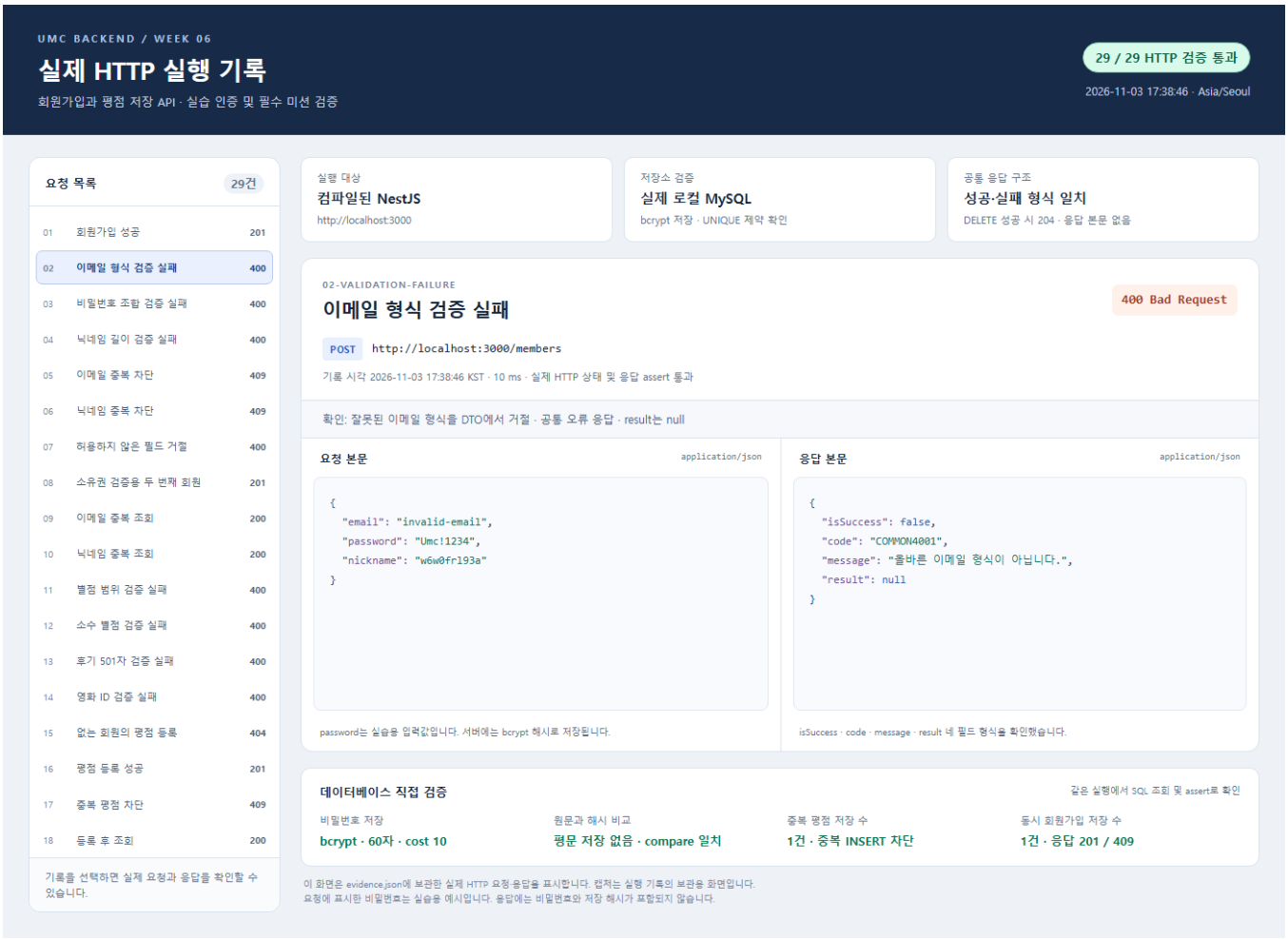
POST /members → 201 Created

{"isSuccess":true,"code":"COMMON201","message":"회원가입에 성공했습니다.","result":{"memberId":1,"email":"week6-w0frl93@example.com","nickname":"w6w0frl93a"}}

HTTP 201로 회원이 생성됐으며 응답 result에는 memberId, email, nickname만 있다. password와 bcrypt 해시는 반환하지 않았다.

핵심 코드: src/movie/member.controller.ts, member.service.ts, dto/create-member.dto.ts

입력 검증 실패



실제 HTTP 실행 기록 화면 | 2026-11-03 17:38:46 (Asia/Seoul)

확인 결과

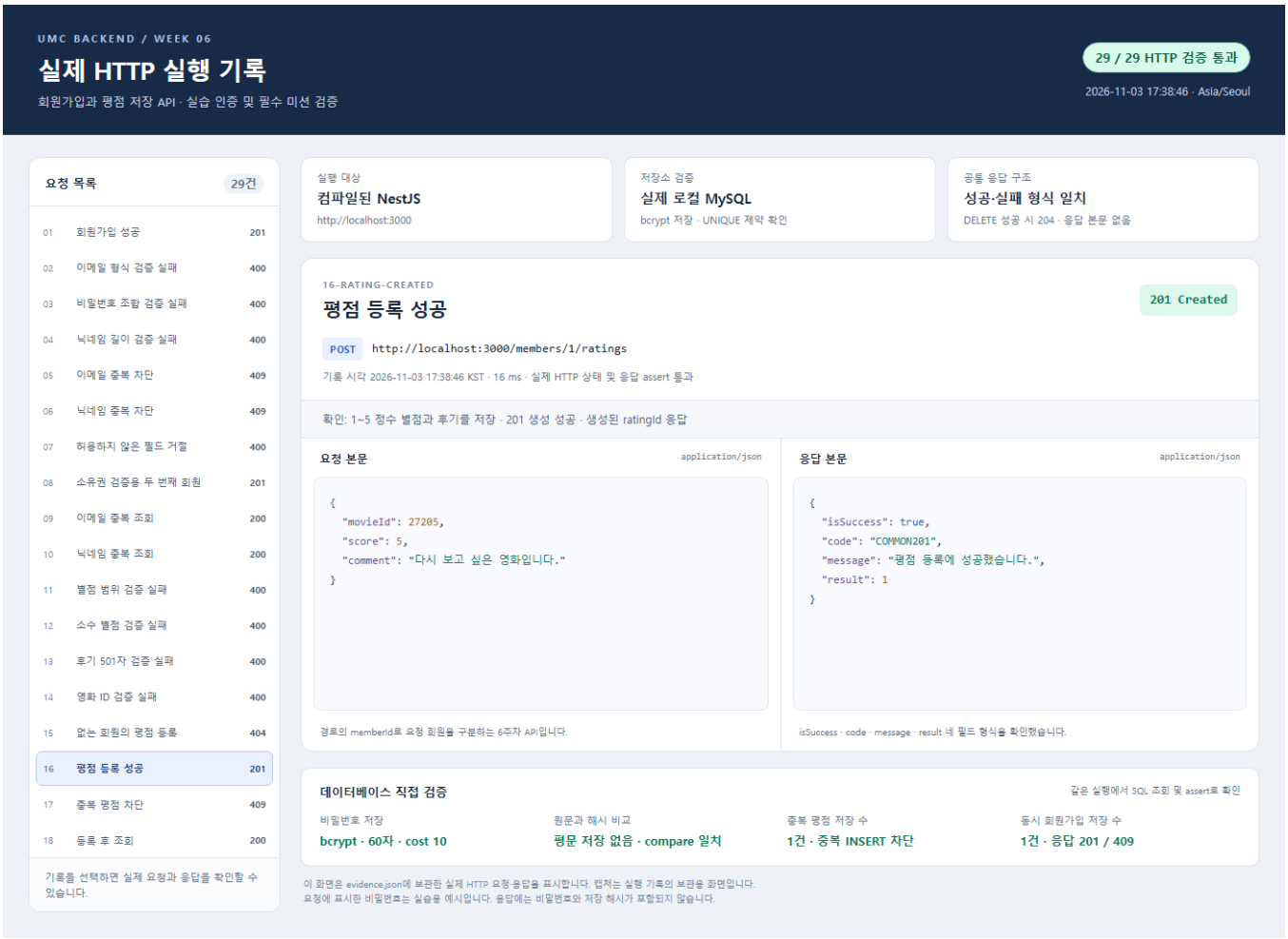
POST /members → 400 Bad Request

{"isSuccess":false,"code":"COMMON4001","message":"올바른 이메일 형식이 아닙니다.","result":null}

잘못된 이메일은 DB 저장 전 DTO 검증에서 HTTP 400으로 거절했다. 실패 응답도 isSuccess, code, message, result 구조를 사용하고 result는 null이다.

별점 6점·소수, 영화 ID 0, 후기 501자, 약한 비밀번호, 잘못된 닉네임 및 추가 필드도 실제 HTTP 요청에서 400으로 확인했다.

평점 등록과 수정



실제 HTTP 실행 기록 화면 | 2026-11-03 17:38:46 (Asia/Seoul)

등록 결과

POST /members/1/ratings → 201 Created

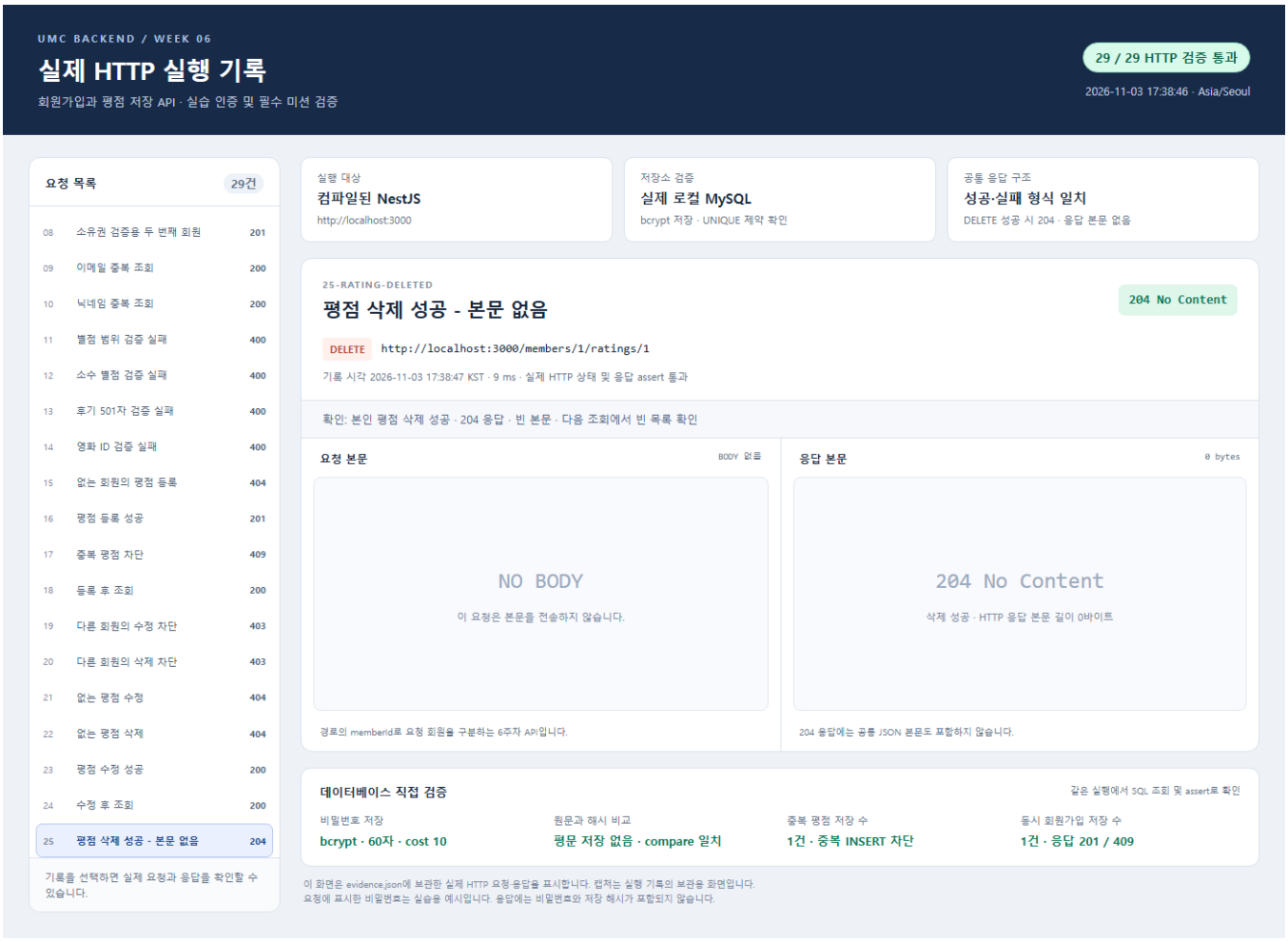
{"isSuccess":true,"code":"COMMON201","message":"평점 등록에 성공했습니다.","result":1}

회원의 TMDB 영화 ID 27205에 score 5와 후기를 등록했다. movieId는 외부 ID이며 로컬 movie 테이블 FK를 요구하지 않는다.

수정 후 재조회

PATCH 요청은 HTTP 200, result null을 반환했다. 이후 GET으로 score 4, comment '수정한 후기입니다.'로 변경된 것을 확인했다. 수정 성공 캡처도 별도 제공한다.

평점 삭제와 삭제 후 조회



실제 HTTP 실행 기록 화면 | 2026-11-03 17:38:46 (Asia/Seoul)

확인 결과

DELETE /members/1/ratings/1 → 204 No Content

```
(empty body)
```

204 응답은 HTTP 규칙대로 JSON 본문을 보내지 않는다. 삭제 뒤 GET의 result는 빈 배열이며, 같은 평점을 다시 삭제하면 404를 반환했다.

GET /members/1/ratings → 200 OK

```
{"isSuccess":true,"code":"COMMON200","message":"평점 조회에 성공했습니다.","result":[]}
```

핵심 코드: src/movie/rating.controller.ts, rating.service.ts

필수 미션 / 400 · 403 · 404 · 409

실패 응답과 데이터 무결성

상황	HTTP	실제 code
이메일 형식·별점 범위 등 검증 실패	400	COMMON4001
다른 회원의 평점 수정·삭제	403	RATING4031
존재하지 않는 회원	404	MEMBER4041
존재하지 않는 평점 수정·삭제	404	RATING4041
이메일 중복	409	MEMBER4091
닉네임 중복	409	MEMBER4092
동일 회원·영화의 중복 평점	409	RATING4091

실제 MySQL 확인

확인 항목	관측값
저장된 비밀번호	bcrypt / 길이 60 / cost 10
원래 비밀번호와 bcrypt.compare	true
평문과 다른 값으로 저장	true
중복 평점 시도 후 저장 행 수	1
동시 회원가입 결과	201 및 409 / 같은 이메일의 저장 행 1개
DB UNIQUE 직접 위반	ER_DUP_ENTRY / errno 1062

DB UNIQUE 제약이 이메일·닉네임·회원과 영화 조합의 중복을 최종 방지한다. 사전 조회가 통과한 동시 요청도 INSERT 시점에는 충돌할 수 있으므로 전역 예외 필터에서 DB 중복 오류를 같은 업무 코드로 변환했다.

원본: output/week6/evidence.json. 중복·소유권·없는 평점 캡처는 screenshots 폴더에 별도로 포함했다.

실제 문제 재현 / 원인 / 수정 / 재검증

트러블슈팅: 경로 ID 자동 변환

이슈

컴파일한 서버에서 GET /members/1e0/ratings가 잘못된 경로 형식인데도 400이 아닌 404 MEMBER4041을 반환했다. 요청이 DTO·파이프 검증을 지나 회원 조회까지 도달한 것을 확인했다.

원인

컨트롤러의 number 메타데이터를 보고 전역 ValidationPipe가 문자열 1e0을 숫자 1로 먼저 바꿨다. 뒤에서 동작하는 PositiveIdPipe는 원래 문자열을 확인할 수 없었다.

해결

경로 인자를 unknown으로 선언해 Number 자동 변환을 피했다. 사용자 정의 파이프가 원래 문자열에 숫자 문자 패턴, 양수, 안전한 정수 범위를 적용한 뒤 서비스에 숫자를 전달한다.

```
@Get(':memberId/ratings')
async getMemberRatings(
  @Param('memberId', PositiveIdPipe) memberId: unknown,
) {
  return success('평점 조회에 성공했습니다.',
    await this.ratings.getRatingsByMember(Number(memberId)));
}
```

재검증

요청	수정 전	수정 후
/members/1e0/ratings	404 MEMBER4041	400 COMMON4001
/members/+1/ratings	-	400 COMMON4001
/members/0x1/ratings	-	400 COMMON4001
/members/1/ratings/1e0 (DELETE)	-	400 COMMON4001

수정 전후 원본: troubleshooting-before.json, troubleshooting-after.json
상세 기록: docs/week6/TROUBLESHOOTING.md

참고: NestJS 공식 Validation 문서, ValidationPipe 공식 소스

제출 시 함께 첨부할 자료

API와 핵심 코드 · 재현 방법

Method / 성공 상태	경로
POST / 201	/members
GET / 200	/members/email/:email
GET / 200	/members/nickname/:nickname
GET / 200	/members/:memberId/ratings
POST / 201	/members/:memberId/ratings
PATCH / 200	/members/:memberId/ratings/:ratingId
DELETE / 204	/members/:memberId/ratings/:ratingId

핵심 코드 링크

제출 가이드의 상대 링크로 전체 소스를 열 수 있다. ZIP에도 동일한 소스 구조를 포함했다. GitHub 제출 시에는 업로드한 커밋의 같은 파일 경로로 링크하면 된다.

```
src/movie/member.service.ts
src/movie/dto/create-member.dto.ts
src/movie/rating.controller.ts
src/movie/rating.service.ts
src/common/configure-app.ts
src/common/http-exception.filter.ts
sql/week6-schema.sql
test/movie.e2e-spec.ts
```

실행 순서

```
npm install
# .env.example을 참고해 로컬 MySQL 환경 설정
npm run week6:db
npm run build
npm run start:prod
# 다른 터미널에서 실제 HTTP 검증
npm run week6:verify
```

Postman Collection과 Environment를 가져온 뒤 순서대로 실행한다. 컬렉션은 테스트 회원 2명을 만들고, 평점을 등록·수정·삭제한다. 실제 HTTP 검증 스크립트는 동시 요청 검증용 회원 1명을 추가로 남긴다. 회원 삭제는 이번 미션 범위에 없다.

외부 제출 사이트 업로드와 GitHub 게시를 수행한 자료는 아니다. 스터디 모임 사진이 필요한 경우 실제 촬영 사진을 별도로 첨부한다.